

Projet React / TypeScript

Réseau social — type Instagram

L3 / B3 — Sprint fullstack

Thomas Diaconu

ESIEE-IT

Coding Factory

Le contexte

Vous rejoignez une équipe. Le backend **existe déjà**.

- Front : **React + TypeScript / JavaScript**, à construire
- Back : **Node.js + TypeScript**, repo fourni
- Groupes de 4 à 5
- 3,5 jours d'implémentation

Vous ne partez pas d'une page blanche.

Le repo fourni — situation réelle

Le backend a été **généré par une IA** par un développeur pressé, puis mergé sans revue.

Il tourne. Il répond. Il fait globalement ce qu'on lui demande.

Il n'est **pas** propre :

- typage relâché, tout dans quelques gros fichiers
- des contrôles de sécurité incomplets
- des choix qui ne tiennent pas à 500 posts
- aucun test, aucune documentation

C'est exactement ce que vous trouverez en arrivant en entreprise.

Vous êtes responsables de ce code **comme si vous l'aviez écrit**.

Il n'y a pas de documentation d'API

Aucun Swagger. Aucun contrat. Aucune spec.

Pour savoir ce que renvoie un endpoint :

- **lisez le code**
- appelez-le et regardez la réponse
- vérifiez le schéma

À retenir

C'est la compétence la plus utile de votre vie professionnelle : comprendre un code que vous n'avez pas écrit.

Livrable 1 — La revue

Avant d'écrire la moindre ligne de front. Un fichier `AUDIT.md`.

Livrable — 30 minutes, J1

1. **Qu'est-ce qui est dangereux ?** données exposées, contrôles d'accès manquants
2. **Qu'est-ce qui ne tiendra pas à 500 posts ?** volumétrie, requêtes, format des réponses
3. **Comment je découperais ce code ?** ce que vous feriez avec une semaine de plus

Court et précis. Fichier, ligne, problème, correction envisagée.

Règle de refactoring

Vous corrigez exactement 3 catégories de défauts :

1. Les failles de sécurité
2. Ce qui empêche une story du socle de fonctionner
3. Ce que vous touchez de toute façon pour livrer

Attention

Réécrire le backend n'est pas le projet. Livrer un front qui tient l'est. En entreprise, on ne refactorise pas ce qu'on n'a pas le temps de tester.

Contraintes de code

Partout

- **TypeScript strict**, typage explicite aux frontières
- **Aucun** `any` . `unknown` + vérification quand le type est incertain
- **Aucun** `as` pour faire taire le compilateur
- Nommage explicite, en anglais, cohérent front / back

Back

- Une classe = une responsabilité, **~150 lignes maximum**
- Une méthode fait **une seule chose**, nom qui la décrit
- Commentaire **uniquement** quand le *pourquoi* n'est pas évident

Front

- Un composant = un rôle. Si vous scrollez, découpez.

Definition of Done

Une story n'est **pas** terminée sans :

- [] Typage strict respecté, lint vert
- [] Les 4 états UI gérés : **loading** / **error** / **empty** / **success**
- [] Erreurs API affichées à l'utilisateur, jamais un écran blanc
- [] Données API **validées** avant usage
- [] Sécurité vérifiée **côté back** (ex : cacher un bouton + protéger route)
- [] Passée en PR, relue par un autre membre, mergée sur `main`

Socle obligatoire

S1 → S8 — Huit stories, toutes attendues, toutes avec la DoD

S1 — Inscription

User Story

En tant qu'utilisateur, je veux m'inscrire, afin de créer un compte.

Critères d'acceptation

- Formulaire : email, mot de passe, nom d'utilisateur
- Champs contrôlés, validation front
- Erreurs API affichées champ par champ
- Mot de passe jamais affiché, **jamais renvoyé par l'API**
- Lien vers la connexion

S2 — Connexion et session

User Story

En tant qu'utilisateur, je veux me connecter et rester connecté.

Critères d'acceptation

- Formulaire email + mot de passe
- Redirection vers le feed
- **Routes protégées** : pas de token, pas d'accès
- Un **401** **déconnecte proprement** et redirige, sans boucle
- Le token survit à un rechargement de page

S3 — Feed

User Story

En tant qu'utilisateur, je veux voir un fil de posts récents.

Critères d'acceptation

- Liste : auteur, contenu, image, date
- Tri décroissant fiable
- **Pagination ou lazy loading** — la base contient ~500 posts
- Les 4 états UI, y compris **liste vide**
- Le chargement d'une page suivante ne fait pas clignoter la liste

S4 — Créer un post

User Story

En tant qu'utilisateur, je veux publier un post avec une image.

Critères d'acceptation

- Contenu texte non vide, longueur maximale validée
- **Image optionnelle avec prévisualisation** avant envoi
- Taille et format contrôlés côté front
- État d'envoi visible, bouton non spammable
- **Feed à jour immédiatement**, sans rechargement
- Erreurs API gérées, formulaire non perdu en cas d'échec

S5 — Détail d'un post et commentaires

User Story

En tant qu'utilisateur, je veux ouvrir un post, le lire et le commenter.

Critères d'acceptation

- Page dédiée, accessible par URL directe
- Navigation depuis le feed
- Liste des commentaires, champ contrôlé, ajout dynamique
- Cas d'un post inexistant géré (404, pas de crash)
- Erreurs API gérées

S6 — Like / Unlike

User Story

En tant qu'utilisateur, je veux liker un post.

Critères d'acceptation

- Bouton like / unlike, compteur à jour
- **Un seul like par utilisateur**, garanti côté back
- **Réaction immédiate à l'écran**, avant la réponse serveur
- **Retour en arrière si l'API échoue**, avec message
- Liker depuis le feed ou depuis le détail donne le même résultat partout

S7 — Profils

User Story

En tant qu'utilisateur, je veux voir mon profil et celui des autres.

Critères d'acceptation

- Une seule page, deux contextes : le mien, celui d'un autre
- Infos publiques, liste des posts de l'utilisateur
- Les actions réservées au propriétaire n'apparaissent que chez lui
- Utilisateur inexistant géré
- Les 4 états UI

S8 — Suppression

User Story

En tant qu'utilisateur, je veux supprimer mes posts et mes commentaires.

Critères d'acceptation

- **Auteur uniquement — vérifié côté backend**
- Confirmation avant suppression
- Mise à jour de l'UI sans rechargement
- Une requête forgée par un autre utilisateur doit être **refusée par l'API**

Attention

Testez-le avec un client HTTP, pas seulement avec votre interface.

Catalogue de défis

Optionnels, à points — mieux vaut un défi bien fait que trois bâclés

Défis — liste

Défi	Intérêt
Retour au feed sans perdre le scroll	cycle de vie, cache
Bannière « nouveaux posts » sans saut	polling, UX
Recherche pilotée par l'URL	<code>searchParams</code> , debounce, annulation
Follow / Unfollow + feed personnalisé	relations, filtrage API
Modifier son profil	formulaire, upload avatar
Modération admin	rôles, autorisation
Tests e2e (Playwright)	scénario complet
CI GitHub Actions	lint + build bloquants
Version mobile	Responsive

Chaque groupe en choisit selon son niveau.



- Une branche par story : `feat/s3-feed`
- **Aucun commit direct sur `main`**
- Toute PR relue par un autre membre avant merge
- Commits courts et lisibles, en anglais
- `node_modules` , `.env` et uploads **jamaïs** commités

L'historique Git est un livrable. Il sera lu.

GenAI

Autorisé et encouragé. Outils : ChatGPT, Claude, Copilot...

Conditions :

- vous comprenez le code
- vous savez l'expliquer, ligne par ligne
- aucun plagiat

Le backend fourni **est** du code généré par IA. Vous voyez ce que ça donne quand personne ne le relit.

Vous êtes responsables de **100 %** du code rendu.

Évaluation

Barème

	Points
Socle S1 → S8	7
Revue + qualité du code et du typage	4
Défis	3
Git	3
Présentation	3

Follows-up

- **Bug injecté** : je casse quelque chose dans votre repo, vous debuggez devant moi
- Questions sur votre propre code
- Questions React / TypeScript
- Expliquer des lignes précisément



Repères de planning

Moment	Attendu
J1 soir	Revue du back, setup frontend
J2	S1, S2, S3
J3	S4, S5, S6, S7
J4	S8, Socle complet, DoD respectée
J4 soir	Défis, préparation de la review

Attention

Si le socle n'est pas fini J3 soir, ne commencez aucun défi.

 **Bonne chance**

Soignez votre travail !